

# Proxy: A Pointer-Semantics-Based Polymorphism Library

Document #: P3086R5 [[Latest](#)] [[Status](#)]  
Date: 2025-10-06  
Project: Programming Language C++  
Audience: LEWGI  
LEWG  
Reply-to: Mingxin Wang  
<[mingxwa@outlook.com](mailto:mingxwa@outlook.com)>

## Contents

- [1 History](#)
  - [1.1 Changes from P3086R4](#)
  - [1.2 Changes from P3086R3](#)
  - [1.3 Changes from P3086R2](#)
  - [1.4 Changes from P3086R1](#)
  - [1.5 Changes from P3086R0](#)
- [2 Introduction](#)
- [3 Motivation and scope](#)
  - [3.1 Problem statement](#)
  - [3.2 Proposed approach](#)
  - [3.3 Goals](#)
  - [3.4 Implementation status](#)
  - [3.5 Informal type-theoretic framing](#)
  - [3.6 Comparison with virtual functions](#)
    - [3.6.1 Memory layout overview](#)
    - [3.6.2 Indirect call sequence comparison](#)
    - [3.6.3 Performance and code generation highlights](#)
  - [3.7 Lifetime and accessibility overview](#)
  - [3.8 Summary of benefits](#)

- 4 Impact on the standard
- 5 Considerations and design decisions
  - 5.1 Primary design driver: pointer semantics to decouple lifetime strategy from polymorphic calls
  - 5.2 Why the name “proxy”
  - 5.3 Library facility, not a language feature
  - 5.4 Facade model and composition
  - 5.5 Performance as a first-class consideration
  - 5.6 Scoping strategy
  - 5.7 Freestanding importance and proof
  - 5.8 Boilerplate vs future automation
  - 5.9 Constraint levels and predictable storage
  - 5.10 Substitution & composition details
  - 5.11 Views (observation) and weak ownership
  - 5.12 Diagnostics and compile-time cost
  - 5.13 Reflection surface positioning
  - 5.14 Summary of design trade-offs
- 6 Technical specifications
  - 6.1 Feature test macro
  - 6.2 Named requirements
    - 6.2.1 *ProOverload*
    - 6.2.2 *ProDispatch*
    - 6.2.3 *ProBasicConvention*
    - 6.2.4 *ProConvention*
    - 6.2.5 *ProBasicReflection*
    - 6.2.6 *ProReflection*
    - 6.2.7 *ProBasicFacade*
    - 6.2.8 *ProFacade*
    - 6.2.9 *ProAccessible*
  - 6.3 Header synopsis
  - 6.4 General wording
    - 6.4.1 Enum class `constraint_level`

- 6.4.2 Concept facade
- 6.4.3 Concept proxiable
- 6.4.4 Class template proxy\_indirect\_accessor
- 6.4.5 Class template proxy
- 6.4.6 proxy\_invoke
- 6.4.7 proxy\_reflect
- 6.4.8 Class substitution\_dispatch
- 6.4.9 observer\_facade and alias template proxy\_view
- 6.4.10 weak\_facade and alias template weak\_proxy
- 6.4.11 Equality and swap
- 7 Acknowledgments
- 8 Summary

### *Abstract*

proxy is a proposed `<memory>` facility providing pointer-semantics-based runtime polymorphism. A facade is a compile-time description of operations (conventions), reflection hooks, and storage / lifetime constraints. A proxy`<F>` holds a pointer-like witness plus facade metadata; concrete types remain layout- and ABI-stable and need not participate in an inheritance hierarchy. The design yields: (1) non-intrusive post-hoc interface composition, (2) an owning / observing / weak handle spectrum sharing one callable surface, (3) explicit capability downgrades via substitution without reconstruction or allocation, (4) predictable high-performance dispatch paths (0–1 dependent loads before the indirect branch, optional metadata embedding, trivial relocation opportunities), and (5) precise concept-based diagnostics.

## § 1 History

### § 1.1 Changes from P3086R4

- Reworked the introduction and motivation to present the proposal from a type theoretic perspective, clarify the pointer centric model, and articulate the relationship with virtual dispatch used in the current standard.
- Integrated `substitution_dispatch`, `proxy_view` together with `observer_facade`, and `weak_proxy` together with `weak_facade` into the wording and rationale informed by implementation practice.
- Removed `access_proxy`.
- Updated the comparative analysis with virtual functions, smart pointer based wrappers, and bespoke vtable based designs to reflect current committee feedback.

- Clarified the intended sample surface of the proposal by removing dependence on helper macros that live in the reference implementation but are not proposed for standardization.

## § 1.2 Changes from P3086R3

- Updated the wording section to align with the Proxy library implementation version 4.0.0.
  - Added type `facade_aware_overload_t`.
  - Revised the *ProOverload* requirements.
  - Revised constraints on the class template `proxy`. function templates `access_proxy`, `proxy_invoke`, `proxy_reflect`.

## § 1.3 Changes from P3086R2

- Revised the Motivation section.
- Added 9 named requirements (*ProOverload*, *ProDispatch*, *ProBasicConvention*, *ProConvention*, *ProBasicReflection*, *ProReflection*, *ProBasicFacade*, *ProFacade*, *ProAccessible*).
- Added class template `proxy_indirect_accessor`.
- Changed the definition of `proxy::invoke()` and `proxy::reflect()` into free functions `proxy_invoke()` and `proxy_reflect()`.
- Added accessibility support to `proxy` (the *ProAccessible* requirements and function template `access_proxy()`).
- Added `proxy::operator bool()`, `proxy::operator->()` and `proxy::operator*`.
- Changed the definition of `std::swap(proxy, proxy)` into a friend function.
- Removed the Appendix section.

## § 1.4 Changes from P3086R1

As per review comments from LEWGI in Tokyo,

- Removed function template `make_proxy` from the proposed wording.
- Updated the wording of `concept facade`, allowing tuple-like types in the definition of a facade or dispatch.
- Revised the semantics of `concept facade` to allow fallbacks in the invocation of a dispatch.
- Moved the proposed location of the library from a new header to `<memory>`.
- Added a section for freestanding specifications in section 6.

- Added discussion comparing with P3019R6 in section 5.
- Added discussion of ordering and hash support in section 5.
- Added another section for open questions.
- In the appendix, added specification of another two helper macros `PRO_DEF_MEMBER_DISPATCH_WITH_DEFAULT` and `PRO_DEF_FREE_DISPATCH_WITH_DEFAULT`.

## § 1.5 Changes from P3086R0

- Added support for `noexcept` in the abstraction model and updated the `noexcept` clause of `proxy::invoke` and `proxy::operator()`.
- Removed `concept basic_facade` and the constraints on the class template `proxy` to allow more potential optimizations in code generation.

## § 2 Introduction

This paper proposes standard library support for pointer-semantics-based runtime polymorphism. The facility consists of the following *standardization scope* components:

- class template `proxy` – the pointer-semantics handle mediating polymorphic calls
- enum class `constraint_level`
- concepts `facade` and `proxiable`
- class template `facade_aware_overload_t`
- function templates `proxy_invoke` and `proxy_reflect`
- class `substitution_dispatch`
- adaptor `facades` / aliases `observer_facade` / `proxy_view` and `weak_facade` / `weak_proxy`
- nine named requirements (*ProOverload*, *ProDispatch*, *ProBasicConvention*, *ProConvention*, *ProBasicReflection*, *ProReflection*, *ProBasicFacade*, *ProFacade*, *ProAccessible*)

Together these describe a minimal substrate for expression-oriented runtime polymorphism executed strictly through ordinary pointer semantics. No inheritance or mandatory dynamic allocation is required. The abstraction has shipped in production (including inside Microsoft Windows) since 2022.

Out of scope for this initial paper (but shipped in existing implementations) are higher level facade construction helpers, additional skills (formatting, RTTI extensions, slim layout presets, etc.), and convenience factory functions beyond those required to motivate the core. Excluding them keeps the initial wording minimal and focused on the foundational model; they can be proposed incrementally once consensus on the substrate is reached.

## § 3 Motivation and scope

### § 3.1 Problem statement

Existing options for runtime polymorphism each impose a different *coupling axis*:

| Existing approach                                     | Primary coupling                                            | Typical costs                                                  | Gaps for modern use                                                                     |
|-------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Virtual functions                                     | Interface <-> object layout & ABI (vptr), inheritance graph | Intrusive base classes, fixed surface, per-object vptr storage | Hard to refactor; ownership orthogonal; extending capability requires hierarchy surgery |
| Ad-hoc vtable wrappers                                | Wrapper <-> bespoke table layout                            | Repeated machinery, uneven capability/reflection               | Ecosystem fragmentation; inconsistent substitution semantics                            |
| Single-purpose type erasure (std::function, std::any) | Wrapper <-> specific call pattern or value semantics        | Often heap allocation; narrow operation set                    | Cannot compose heterogeneous operation families or ownership modes                      |

Users still need a facility that simultaneously provides:

1. Non-intrusive type erasure over arbitrary existing types (standard, 3rd-party, aggregates) with no required base class.
2. An explicit lifetime spectrum (owning, observing, weak) unified under one invocation surface.
3. Allocation-free capability refinement and downgrades (substitution) without wrapper reconstruction.
4. Composition of independently specified operation families (member, free, operator, conversion, CPO) while keeping natural call syntax.
5. Static reflection hooks and precise diagnostics, remaining freestanding-friendly.
6. Predictable performance characteristics (instruction count, indirection depth, binary size) with portable optimization triggers.

### § 3.2 Proposed approach

A **facade** is a compile-time description of admissible operations (“conventions”), associated reflection hooks, and storage / lifetime constraint parameters (max size, alignment, copy / relocate / destroy levels). A `proxy<F>` pairs a pointer-like witness with facade metadata. Adaptors (`observer_facade`, `weak_facade`) systematically rewrite an existing facade to strip owning semantics or introduce weak locking while preserving the call surface where valid.

`substitution_dispatch` provides explicit conversions (including capability downgrades) between proxies defined by a facade; it is not restricted to structural subset relations of convention sets. It guarantees null propagation and may exploit trivial relocatability to avoid invoking user move constructors.

Concepts and named requirements precisely specify when a type can participate. This aligns with a well-understood existential pattern: informally, `proxy<F>` is an existential wrapper over the family of types modeling *ProFacade* relative to `F`.

## § 3.3 Goals

- Pointer-centric, non-intrusive runtime polymorphism
- Deterministic, explicit lifetime across owning / observing / weak handles
- Extensible composition of conventions (member / free / operator / conversion / CPO forms)
- Substitution to support capability refinement and API versioning
- Compact wording; forward compatible with executors / senders, ranges, reflection

## § 3.4 Implementation status

The open-source implementation (<https://github.com/microsoft/proxy>) targets C++20 and later and is continuously tested on GCC, Clang, MSVC, NVIDIA HPC, and Intel oneAPI across Windows, Linux, and macOS. The wording tracks version 4 of the shipping library.

## § 3.5 Informal type-theoretic framing

Runtime polymorphism is modeled as morphisms between pointer witnesses. A facade enumerates morphism families (dispatch conventions) plus constraints ensuring the witness’s storage satisfies the required lifetime and layout properties. The named requirements (*ProOverload*, *ProDispatch*, etc.) guarantee consistent metadata formation; `facade_aware_overload_t` captures recursively facade-aware overload sets. Because witnesses are ordinary pointers (or pointer-sized handles), the existing object model and aliasing rules apply unchanged. Compared to the fixed product of function pointers embedded per object in traditional virtual tables, a facade is a *post-hoc* composition boundary: capability downgrades and adaptations are expressed explicitly via `substitution_dispatch` rather than implicit hierarchy relationships.

## § 3.6 Comparison with virtual functions

Memory layout and indirection:

- Virtual dispatch embeds a vptr (or more with multiple/virtual inheritance) in every dynamic object; the callable surface is a fixed product decided at class definition. Upcasts are pointer adjustments; calls load the vptr then an entry.
- proxy<F> keeps the implementation object's layout unchanged. The proxy stores a pointer-like witness plus facade metadata (often a pointer to a dispatch table or in-lined table if small). Adding or composing conventions does not require touching the implementation type; different client code can materialize different facades over the same underlying type.

### § 3.6.1 Memory layout overview

The following diagrams contrast a typical single-inheritance virtual layout with a proxy-based arrangement wrapping the same implementation object. The implementation type (“Impl”) is identical in both cases; only the surrounding indirection differs.

Object with virtual functions:

proxy with indirect storage and large metadata:

```
flowchart LR
    subgraph Proxy[proxy]
        WP["(witness ptr)"]
        MD["(metadata)"]
    end
    WP --> OImpl[Impl data]
    MD --> D1["f()"]
    MD --> D2["g()"]
    MD --> D3["h()"]
```

proxy with direct storage and large metadata:

```
flowchart LR
    subgraph Proxy[proxy]
        subgraph WP["witness ptr"]
            OImpl[Impl data]
        end
        MD["(metadata)"]
    end
```

```
MD --> D1["f()"]
MD --> D2["g()"]
MD --> D3["h()"]
```

proxy with direct storage and small metadata:

```
flowchart LR
    subgraph Proxy["proxy"]
        subgraph WP["witness ptr"]
            OImpl["Impl data"]
        end
        subgraph MD["(metadata)"]
            D["f()"]
        end
    end
```

Engineering / evolution:

- Virtual hierarchies force early ABI and inheritance decisions; refactoring surface or storage later can be costly.
- Facades can be extended by defining new ones that compose existing conventions. Substitution allows code expecting an older (smaller) facade to accept newer objects without wrapper reconstruction.

Memory management and heterogeneity:

- With virtual bases, ownership concerns are orthogonal (raw pointers, `unique_ptr`, `shared_ptr`), each layered externally; in-place alternatives require separate wrappers.
- proxy can hold any pointer-like witness object meeting size/alignment/constraint requirements, including custom allocator-aware handles, offset/arena pointers, or small in-place storage (via facilities that are part of the deployed library but out of scope for standardization here). Borrowing (`proxy_view`) and weak (`weak_proxy`) forms reuse the same facade definitions; algorithms can negotiate the weakest acceptable capability via substitution.

Operation surface: - Virtual dispatch is member-function-centric; free functions and many operators typically need auxiliary patterns (ADL helpers, CRTP, friend thunks, or visitors).  
 - Facades enumerate conventions regardless of whether they correspond to member, free, operator, or conversion forms; accessors keep call syntax natural.

Summary:

| Concern               | Virtual dispatch                     | This proposal                               |
|-----------------------|--------------------------------------|---------------------------------------------|
| Intrusiveness         | Requires base class & altered layout | Works with existing, non-hierarchical types |
| Surface extensibility | Fixed at definition                  | Post-hoc facade composition                 |

| Concern               | Virtual dispatch             | This proposal                                     |
|-----------------------|------------------------------|---------------------------------------------------|
| Ownership spectrum    | External smart pointers      | Integrated owning / observing / weak forms        |
| Layout impact         | vp <sub>ptr</sub> per object | Object layout unchanged                           |
| Heterogeneous storage | Additional wrappers          | Any pointer-like witness (constraints permitting) |
| Null handling         | Convention-dependent         | Uniform null propagation                          |

### § 3.6.2 Indirect call sequence comparison

Key dynamic call path differences versus virtual dispatch:

1. Embedded metadata option: for small facades the proxy can embed one (or a small fixed number of) function pointers directly in the proxy object. The call site then performs a single indirect branch (no preceding metadata pointer load). A virtual call must always (a) load the vp<sub>ptr</sub> then (b) load the slot before branching.
2. Table (non-embedded) metadata: when the facade is larger, the proxy holds a pointer to a dispatch table. The call path becomes one metadata load + one branch (still one fewer dependent load than the typical virtual sequence on the architectures examined).
3. Late witness dereference: the call site need not first load the implementation (“this”) pointer; the dispatch stub (or target) receives the proxy itself and can load the underlying object pointer lazily. This removes instructions at each call site; the cost (a load of the stored witness) migrates into the dispatch stub, which may be inlined or shared across call sites.
4. Arbitrary witness pointer: the stored witness may be a raw pointer, small owning handle, arena / offset pointer, custom allocator handle, or a small in-place object wrapper. Virtual dispatch always presumes an object with an embedded vp<sub>ptr</sub> at a fixed layout.

Trade-offs: Embedding shrinks per-call instruction count but increases proxy size if many conventions are inlined; implementations typically switch to a table form after a small threshold. Late witness dereference slightly lengthens the dispatch stub (it must recover the implementation pointer) but keeps every call site minimal.

Sequence diagrams for a single convention `draw()`:

Embedded metadata (0 loads at call site):

```
sequenceDiagram
    participant C as Caller
    participant P as Proxy (embedded)
    participant S as Stub
    participant I as Impl
    C->>P: call draw()
```

```
P-->>S: fn ptr (field)
C->>S: indirect branch
S->>I: load witness & invoke
```

Table metadata (1 load + branch):

```
sequenceDiagram
    participant C as Caller
    participant P as Proxy (table)
    participant M as Table
    participant I as Impl
    C->>P: call draw()
    P->>M: load entry
    M-->>I: fn ptr
    C->>I: indirect branch
```

Virtual dispatch (2 loads + branch):

```
sequenceDiagram
    participant C as Caller
    participant O as Object
    participant VT as VTable
    C->>O: call draw()
    O->>VT: load vptr
    VT-->>O: load slot
    C->>O: indirect branch
```

Observed effects in tight polymorphic loops (microbenchmarks and production traces):

- Fewer serialized pointer chases on the critical path (0 or 1 vs 2) reduce latency variance and may aid branch prediction.
- Smaller per-call instruction footprint can reduce I-cache pressure and code size.
- Work hoisted into shared dispatch stubs amortizes better when many call sites target the same facade.
- When inlining completely eliminates the stub, the callee may resemble an ordinary free function operating on a witness pointer it loads once.

Potential downsides / neutrality points:

- If the dispatch stub is cold and not inlined, one extra load to recover the implementation pointer appears inside the stub (not at each call site), partially offsetting savings.
- Embedding multiple pointers can increase proxy size; implementations tune thresholds empirically.
- Some architectures/speculative execution scenarios may already hide one of the virtual loads; real speedups depend on surrounding memory pressure.

### § 3.6.3 Performance and code generation highlights

We revalidated the call path analysis with a minimal experiment: a facade containing a single const, non-throwing void member-like operation on a trivially movable implemen-

tation type (two integers plus padding), compared against a classic abstract base with one virtual function and a similarly sized derived type. Two functions perform one dynamic call each: one through an owning proxy, one through `std::unique_ptr<Base>`. Both allocate the concrete object dynamically; the proxy form populates embedded or table metadata depending on configuration.

Clang 21.1.0 (-O3 -std=c++20 -DNDEBUG) generated on three architectures:

| Pattern           | x86-64                                      | AArch64                                            | RV64                                               |
|-------------------|---------------------------------------------|----------------------------------------------------|----------------------------------------------------|
| Proxy call site   | jmp [rdi+8]                                 | ldr x1,<br>[x0,#8]<br>br x1                        | ld a5,8(a0)<br>jr a5                               |
| Virtual call site | mov rdi,[rdi]<br>mov rax,[rdi]<br>jmp [rax] | ldr x0,[x0]<br>ldr x8,[x0]<br>ldr x1,[x8]<br>br x1 | ld a0,0(a0)<br>ld a1,0(a0)<br>ld a5,0(a1)<br>jr a5 |

The embedded-metadata configuration reduces the call site to a single indirect branch. When more entries require a table pointer, the observed forms are:

| Strategy                  | x86-64 (illustrative)                             | Loads before branch |
|---------------------------|---------------------------------------------------|---------------------|
| Embedded function pointer | jmp [rdi+8]                                       | 0                   |
| Table pointer in proxy    | mov rax,[rdi]<br>jmp [rax+offset] (or equivalent) | 1                   |
| Virtual dispatch          | mov rax,[rdi]<br>mov rax,[rax]<br>jmp [rax+slot]  | 2                   |

Construction code generation shows the proxy populating function pointer fields directly, while the virtual path writes a vptr into the newly allocated object. The proxy's flexibility appears in that the stored witness could instead be:

- A small in-place object (when size / alignment & constraint levels permit)
- A custom arena / offset pointer (no allocation at wrapper construction)
- A shared / intrusive reference handle

Representative microbenchmark summary (values are baseline time divided by proxy time; >1 means proxy faster):

| <b>Scenario<br/>(median)</b>           | <b>MSVC x64</b> | <b>GCC x64</b> | <b>Clang x64</b> | <b>Apple<br/>Clang<br/>ARM64</b> |
|----------------------------------------|-----------------|----------------|------------------|----------------------------------|
| Indirect call<br>(small<br>object)     | 3.6×            | 1.45×          | 1.7×             | ~1.0×                            |
| Indirect call<br>(large<br>object)     | 2.9×            | 1.16×          | 1.17×            | 1.1×                             |
| Lifetime vs<br>unique_ptr<br>(small)   | 5.7×            | 5.1×           | 5.3×             | 4.4×                             |
| Lifetime vs<br>shared_ptr<br>(no pool) | 7.4×            | 6.1×           | 5.9×             | 5.8×                             |

Representative; full data including variance, allocator sensitivity, and alternative metadata packing strategies remain in the public benchmark suite and the blog post: [Analyzing the Performance of the “Proxy” Library](#).

Interpretation (representative, not normative):

- The largest relative gains appear where instruction count and dependent loads dominate (tight loops over small polymorphic objects on x86-64).
- ARM64 often partially hides one of the virtual loads via memory-level parallelism; gains narrow accordingly.
- Lifetime operations benefit primarily because proxy allows small buffer optimization (inline storage inside a pointer-like witness) while keeping the same polymorphic surface; trivial relocation further helps but is secondary.
- Some workloads dominated by heavy callee bodies (compute bound) see negligible difference; the expectation is that for common call-heavy cases the proxy path is faster or equal, and users can tune performance by adjusting facade parameters (max size, alignment, copyability/relocatability/destructibility constraint levels).

Cost considerations / neutrality:

- Dispatch stub size and I-cache effects may offset wins if many rarely executed conventions are embedded; implementations spill to a table adaptively.
- Very large facades that always require a table reduce the gap; the design still avoids the per-object vptr cost.
- The flexibility to choose allocation (or not) at facade instantiation time is itself a performance lever: users can match storage to locality / lifetime requirements without altering the polymorphic surface.

Across the shown targets the proxy path often removes one or two dependent pointer dereferences per dispatch relative to the minimal virtual pattern (for these layout strategies). Eliminating even a single repeated load in a hot indirect call site compounds: tighter critical path, smaller instruction footprint, and reduced I-cache pressure. Effects vary with architecture and workload; compute-bound bodies may mask differences.

## § 3.7 Lifetime and accessibility overview

- `proxy<F>` may store any pointer-like witness object (raw pointer, custom smart pointer, allocator / arena handle, offset pointer, tagged pointer, small in-place object via helper facilities, etc.) that satisfies the facade's constraint levels and implements the required conventions / reflections. The standard proposal scopes only the core handle; helper creation functions (e.g. `make_proxy`, `allocate_proxy`, `make_proxy_inplace`, `make_proxy_shared`) exist in the deployed library but are not proposed here.
- `proxy_view<F> = proxy<observer_facade<F>>`; conventions requiring ownership are removed; substitution to a view never strengthens ownership.
- `weak_proxy<F> = proxy<weak_facade<F>>`; adds `lock()` returning `proxy<F>`; substitution rewrites return types to weak forms to avoid accidental strengthening.
- *ProAccessible* defines accessor templates; `substitution_dispatch` ensures null propagation and enables trivial relocation semantics.

## § 3.8 Summary of benefits

The facility yields a coherent pointer-centric model that:

1. decouples polymorphic interface description from object layout,
2. unifies owning / observing / weak handles under one facade,
3. supports composable operation sets beyond member functions,
4. enables allocation-free capability refinement,
5. produces clear diagnostics via concept-based named requirements.

## § 4 Impact on the standard

The proposal adds a cohesive set of facilities to `<memory>`; no core language changes are required. Only the components listed above (and the two adaptors demonstrated as essential) are in scope. A feature test macro `__cpp_lib_proxy` gates adoption.

Interoperability: existing facilities (`std::function`, `std::any`, polymorphic allocators, sender/receiver, ranges) can layer over proxy by defining facades matching their operational subsets, avoiding duplicated type erasure machinery. Implementations that already ship a similar abstraction can provide a thin compatibility facade that forwards to their in-

ternal representation (or vice versa) using ordinary headers; no special inline-namespace remapping is required beyond the usual vendor versioning practices.

Freestanding friendliness and header-only structure permit incremental vendor adoption. The type-theoretic framing addresses prior teachability concerns; the performance and code generation evidence addresses cost concerns. The design is intentionally minimal yet extensible.

## § 5 Considerations and design decisions

This section collects the highest impact design drivers first, then details secondary but still relevant considerations. The overarching theme: maximize runtime performance and flexibility by keeping the polymorphic abstraction strictly in the library domain and aligned with ordinary pointer semantics.

### § 5.1 Primary design driver: pointer semantics to decouple lifetime strategy from polymorphic calls

The single most important decision is to ground the abstraction in *pointer semantics* rather than object layout changes (vp<sub>tr</sub> injection) or mandatory heap indirection. A proxy<F> holds a pointer-like *witness* plus facade metadata; the underlying concrete object remains layout- and ABI-stable. This decoupling yields several concrete benefits:

- Lifetime model independence: owning, observing, weak, arena/offset, PIMPL, custom allocator handles, intrusive or non-intrusive forms all become simple choices of pointer-like witness type, not different wrapper classes.
- Small / in-place optimization is explicit and bounded by the facade constraint parameters (`max_size`, `max_align`, constraint levels for copy / relocation / destruction). Implementations can select direct embedding, an SBO buffer, or a raw pointer with no semantic ambiguity.
- Call path minimization: dispatch sites eliminate one or two dependent loads relative to the canonical virtual pattern; pointer identity and aliasing rules remain those of the underlying C++ object model.
- Trivial relocation opportunity: when the witness type (e.g. pointer or trivially relocatable handle) satisfies the facade’s constraint level, substitution and movement can be implemented as a “trivial relocation” with no user-defined move constructor cost.

Performance concerns (predictable indirection depth, instruction footprint, cache behavior) are *dominant* here; expressivity follows, not precedes, the pointer-centric model.

### § 5.2 Why the name “proxy”

“Proxy” is a familiar word in multiple programming domains describing an intermediary that forwards or mediates operations: JavaScript’s Proxy object (ECMAScript 6), dynamic

proxies in Java / C#, remote proxies in distributed systems, and classic design patterns. Despite association with networking, the core semantics (an interposed representative controlling access or behavior) match this facility precisely: a proxy<F> *represents* a concrete object via pointer indirection while arbitrating the allowable operations declared by the facade.

Alternative names (“erasure”, “handle”, “witness”, “facade\_handle”) were considered but rejected for one or more reasons:

- “handle” is overloaded in the STL and OS APIs and does not connote polymorphic dispatch.
- “erasure” over-emphasizes mechanism (type erasure) rather than role.
- “witness” is accurate in type theory but unfamiliar to wide audiences.
- “facade” already names the compile-time description; overloading it for the runtime object would blur the separation.

“proxy” therefore offers: (1) cross-language precedent for indirection with behavioral control, (2) teachability (developers already explain design pattern proxies), (3) brevity and suitability as a template identifier, (4) alignment with the extensions (`weak_proxy`, `proxy_view`) that remain semantically clear.

## § 5.3 Library facility, not a language feature

Choosing a library design permits immediate deployment, experimentation, and composition without committing to new core-language constructs or syntax. Specific rationales:

- No new object model rules: witnesses are ordinary pointers / small objects; aliasing, lifetime, and layout remain governed by existing C++ semantics. A language feature altering ABI (like new hidden vptr forms) would be heavier.
- Customizable composition: facades are ordinary types; users can define new conventions and reflections directly (this paper shows the minimal manual form). Helper builder utilities exist in the deployed library but are intentionally out of scope here.
- Evolutionary path: features like reflection-based automatic facade generation or declarative syntax sugar can layer later once consensus exists; the current library subset validates core semantics today.
- Freestanding viability: all components are implementable in a freestanding environment (already demonstrated) without compiler hooks.
- Tooling & modules: compile-time cost mitigation (modules) and improved diagnostics (concepts) arrive for free in library form; a language feature would not inherently simplify those further.

Importantly, none of the performance wins depend on privileged compiler knowledge; they arise from deliberate layout and dispatch design expressible in portable C++.

## § 5.4 Facade model and composition

A facade is a *compile-time algebraic description* of: (a) a set of conventions (dispatchable operation families), (b) optional reflection hooks, (c) storage constraint parameters. One can manually compose capability sets by defining a new facade type whose `convention_`-types / `reflection_types` tuples include (or re-express) those of earlier facades and by tightening constraint constants where desired. (Out-of-scope helper builders in the existing implementation automate this aggregation, but the core model does not depend on them.) Substitution is *declared* via a direct convention whose dispatch type is `substitution_dispatch` producing another proxy<G> (or view / weak variants under adaptation).

Facade anatomy and relationships:

flowchart LR

```
subgraph F[Facade]
  Cs[convention_types: tuple-like]
  Rs[reflection_types: tuple-like]
  MaxSize[max_size: size_t]
  MaxAlign[max_align: size_t]
  Copyability[copyability: constraint_level]
  Relocatability[relocatability: constraint_level]
  Destructibility[destructibility: constraint_level]
end
```

```
subgraph C1[Convention A]
  C1IsDirect[is_direct: bool]
  subgraph C1D[dispatch_type]
    C1A["accessor (optional)"]
  end
  C1Os[overload_types: tuple-like]
end
```

```
subgraph R1[Reflection A]
  R1IsDirect[is_direct: bool]
  subgraph R1R[reflector_type]
    R1A["accessor (optional)"]
  end
end
```

```
Cs --> C1
C1Os --> C1O1[Overload A]
C1Os --> C1O2[Overload B]
Cs --> C2[Convention B]
Rs --> R1
Rs --> R2[Reflection B]
```

Key properties:

- Separation of *what* can be invoked (compile-time facade description) from *how* it is invoked (runtime metadata layout chosen per implementation & constraint levels).
- Extensible algebra: adding a new operation family does not alter existing implementations of unrelated conventions; users define new convention types that still partici-

pate in accessibility via *ProAccessible*.

- Substitution is explicit, not a structural subset guess, allowing non-hierarchical capability refinement (e.g. adapting return types, preserving `noexcept`, preserving null propagation).

## § 5.5 Performance as a first-class consideration

Performance is not an afterthought: the call path structure, trivial relocation, metadata embedding threshold, and constraint level encoding were all chosen to minimize instruction count, dependent loads, and code size in hot polymorphic loops. This reiterates and deepens Motivation: pointer semantics enable these wins by keeping the underlying object opaque yet layout-neutral.

Committee review often asks whether speedups hold in real code. One large reported migration (approx. 120k lines) from pure vtable interfaces (COM-style) to proxy observed 8x-10x improvements in certain scenarios. While part of the gain came from fixing prior design issues (e.g. overuse of `std::dynamic_pointer_cast`), a good abstraction *guides* better performance by making inefficiencies (extra indirections, unnecessary ownership) explicit and cheap to remove.

## § 5.6 Scoping strategy

The proposal intentionally standardizes only the minimal substrate: proxy, facade (and supporting concepts), `substitution_dispatch`, the view and weak adaptors, core named requirements, and `invoke / reflect` free functions. The shipped implementation includes additional helpers (factory functions, facade-construction utilities, skill sets, formatting, RTTI expansions). These are deferred *not* due to immaturity, but to keep initial wording review tractable and focused on the fundamental model. Successful adoption of this subset unblocks follow-on proposals that can add low-boilerplate facade authoring facilities if the committee desires.

## § 5.7 Freestanding importance and proof

Runtime polymorphism is often required in constrained and embedded environments that disallow large library dependencies or RTTI/exception-heavy mechanisms. Every facility here is implementable in a freestanding environment (the reference implementation already compiles freestanding). Decoupling from heavy runtime features increases likelihood of adoption in kernels, drivers, and accelerators where pointer-size-optimized `proxy_view` (observer variant) is valuable.

## § 5.8 Boilerplate vs future automation

Authoring a facade in the *minimal* subset requires spelling out the convention and reflection tuples and the constraint constants explicitly. This is acceptable for an initial standard surface because:

- The expressive power is already higher than traditional virtual interfaces (multiple independent operation families, free/operator/conversion forms, reflection) *without* intrusion into the concrete types.
- Boilerplate is largely mechanical and therefore automatable: future proposals can leverage reflection or standardized helper utilities to auto-derive conventions from function sets.
- Early explicitness accelerates committee understanding of the mechanics (clear mapping to requirement tables) before abstraction layers potentially hide details.

Illustrative contrast (helper utility OUT OF SCOPE vs manual definition in this paper's model):

```
// Hypothetical helper-based form (non-normative here)

struct Streamable : pro::facade_builder
    ::add_convention<pro::operator_dispatch<<"<<", true>, std::ostream&
        (std::ostream& out) const>
    ::build {};
```

```
// Manual minimal form (in-scope pattern)

struct StreamOutDispatch {
    template <class T>
    std::ostream& operator()(const T& self, std::ostream& out) const {
        return out << self;
    }
    template <class P, class D, class... Os> struct accessor; // primary
    template <class P, class D>
    struct accessor<P, D, std::ostream&(std::ostream&) const> {
        friend std::ostream& operator<<(std::ostream& out, const P& self) {
            return std::proxy_invoke<D, std::ostream&(std::ostream&) const>
                (self, out);
        }
    };
};

struct StreamOutConvention {
    using dispatch_type = StreamOutDispatch;
    using overload_types = std::tuple<std::ostream&(std::ostream&) const>;
    static constexpr bool is_direct = false;
};

struct Streamable {
    using convention_types = std::tuple<StreamOutConvention>;
    using reflection_types = std::tuple<>;
    static constexpr std::size_t max_size = 2 * sizeof(void*);
    static constexpr std::size_t max_align = alignof(void*);
    static constexpr auto copyability = std::constraint_level::none;
    static constexpr auto relocatability = std::constraint_level::trivial;
    static constexpr auto destructibility = std::constraint_level::nothrow;
};
```

The manual definition is verbose but straightforward, and serves to validate the facade abstraction before proposing ergonomic helpers.

## § 5.9 Constraint levels and predictable storage

`constraint_level` values in the facade define minimum lifetime operation guarantees. They make optimization conditions *statically checkable* (e.g., enabling trivial relocation paths) and keep the metafunction surface small (single enum rather than a matrix of traits). They also allow users to reason about ABI and object representation when choosing between embedding and indirection.

## § 5.10 Substitution & composition details

`substitution_dispatch` centralizes conversion logic between facades. Advantages of expressing substitution as a dispatch:

- Uniform participation in accessibility (call syntax, noexcept propagation, overload resolution) rather than a hard-coded implicit conversion rule.
- Extensible: new substitution modes (e.g., borrowing conversions, capability downgrades that adapt return types) are added as new overloads without altering the core proxy class template.
- Null propagation is specified once and inherited by all substitutions.

Manual composition of operation sets (by forming a new `convention_types` tuple that lists multiple convention types) enables post-hoc capability aggregation uncommon in inheritance hierarchies where base ordering and diamond rules complicate extension.

## § 5.11 Views (observation) and weak ownership

`proxy_view` (`observer_facade`) and `weak_proxy` (`weak_facade`) share the same operation vocabulary while reducing or altering ownership semantics:

- Views strip owning-only conventions and rewrite substitution returns to remain views, preventing accidental strengthening.
- Weak handles prepend a `lock()` convention and rewrite substitution returns to preserve weakness; this avoids surprising ownership escalation in generic algorithms. Both adaptors rely on the pointer semantic foundation (lifetime never conflated with operation set).

## § 5.12 Diagnostics and compile-time cost

Concept-based named requirements generate *precise* substitution failure messages (missing convention type, incorrect dispatch, constraint level mismatch) that practitioners have reported as shorter and more actionable than errors from monolithic type-erasure wrappers. While proxy is template-heavy, mitigations exist and have been validated:

- Modules materially reduce repeated instantiation cost for large facade families.
- Refactoring into smaller composed facades narrows incremental rebuild scope.

- Future standard reflection could autogenerate boilerplate conventions, further reducing author cost without touching the proven core.

Empirically, compile speed has **not** been a blocker in production deployments (including large internal codebases and Windows components). Therefore compile-time cost should not impede standardization.

## § 5.13 Reflection surface positioning

The current paper keeps the reflection surface minimal but *seated* within the facade model (reflection types in `reflection_types`). This design allows later reflection-centric extensions (introspection, tracing, generic instrumentation) without redesigning or versioning the core runtime handle.

## § 5.14 Summary of design trade-offs

| Aspect                     | Benefit                               | Trade-off / Mitigation                          |
|----------------------------|---------------------------------------|-------------------------------------------------|
| Pointer semantics          | Decoupled lifetime, optimal call path | Requires facade boilerplate (future automation) |
| Facade composition         | Post-hoc capability assembly          | More template instantiations (modules)          |
| Substitution as dispatch   | Extensible, uniform syntax            | Slight indirection for conversion (amortized)   |
| Explicit constraint levels | Static optimization triggers          | Author must choose levels (defaults / guidance) |
| View & weak adaptors       | Unified API across ownership modes    | Additional conceptual surface (documented)      |
| Library (not language)     | Deployable now, evolvable             | No new syntax sugar yet (could follow)          |
| Template-heavy design      | Zero overhead abstractions            | Compilation cost (managed)                      |

Overall, the design prioritizes predictable high-performance polymorphism under explicit user control, deferring non-essential ergonomic helpers until after the foundational model is standardized.

## § 6 Technical specifications

### § 6.1 Feature test macro

In *[version.syn]*, add:

```
#define __cpp_lib_proxy YYYYMMML // also in <memory>
```

The placeholder value shall be adjusted to denote this proposal's date of adoption.

### § 6.2 Named requirements

The following named requirements are introduced. The exposition-only entities mentioned herein have no linkage and are used solely to define requirements.

#### § 6.2.1 *ProOverload*

A type *O* meets the *ProOverload* requirements if, for any facade type *F* that meets *ProBasicFacade*, the exposition-only type `substituted-overload<O, F>` is defined as:

- `O<F>` if *O* is a specialization of `facade_aware_overload_t<OT>`;
- otherwise *O*.

`substituted-overload<O, F>` shall denote one of the following function types where *R* is a return type and *Args...* are zero or more parameter types:

```
R(Args...)
R(Args...) noexcept
R(Args...) &
R(Args...) & noexcept
R(Args...) &&
R(Args...) && noexcept
R(Args...) const
R(Args...) const noexcept
R(Args...) const&
R(Args...) const& noexcept
R(Args...) const&&
R(Args...) const&& noexcept
```

#### § 6.2.2 *ProDispatch*

Given a type *D*, a type *T*, and a type *O* meeting *ProOverload*, let *R* be the return type and *Args...* the parameter types of *O*. A type *D* meets the *ProDispatch* requirements of *T* and *O* if:

- `D()` is well-formed, creates a value of type *D*, and does not throw.
- For every cv/ref/noexcept form of *O*, the invocation expression formed as specified below is well-formed and has the semantics indicated. `args...` denotes a pack of

glvalues of types (possibly cv-qualified) `Args...` where each `args_i` is passed as `std::forward<Args_i>(args_i)`.

Invocation forms (let `v` be an lvalue of type `T`, cv a `const T` lvalue):

| Overload form                                        | Expression                                                                                         | Semantics                                |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------|------------------------------------------|
| <code>R(Args...)</code>                              | <code>INVOKE&lt;R&gt;(D(), v,<br/>std::forward&lt;Args&gt;<br/>(args)...) )</code>                 | Invokes with <code>v</code> ; may throw. |
| <code>R(Args...) noexcept</code>                     | same                                                                                               | Shall not throw.                         |
| <code>R(Args...) &amp;</code>                        | same                                                                                               | May throw.                               |
| <code>R(Args...) &amp;<br/>noexcept</code>           | same                                                                                               | Shall not throw.                         |
| <code>R(Args...) &amp;&amp;</code>                   | <code>INVOKE&lt;R&gt;(D(),<br/>std::move(v),<br/>std::forward&lt;Args&gt;<br/>(args)...) )</code>  | May throw.                               |
| <code>R(Args...) &amp;&amp;<br/>noexcept</code>      | same                                                                                               | Shall not throw.                         |
| <code>R(Args...) const</code>                        | <code>INVOKE&lt;R&gt;(D(), cv,<br/>std::forward&lt;Args&gt;<br/>(args)...) )</code>                | May throw.                               |
| <code>R(Args...) const<br/>noexcept</code>           | same                                                                                               | Shall not throw.                         |
| <code>R(Args...) const&amp;</code>                   | same as <code>const</code> form                                                                    | May throw.                               |
| <code>R(Args...) const&amp;<br/>noexcept</code>      | same as <code>const<br/>noexcept</code> form                                                       | Shall not throw.                         |
| <code>R(Args...) const&amp;&amp;</code>              | <code>INVOKE&lt;R&gt;(D(),<br/>std::move(cv),<br/>std::forward&lt;Args&gt;<br/>(args)...) )</code> | May throw.                               |
| <code>R(Args...) const&amp;&amp;<br/>noexcept</code> | same                                                                                               | Shall not throw.                         |

### § 6.2.3 *ProBasicConvention*

A type `C` meets the *ProBasicConvention* requirements if:

- `C::is_direct` is a core constant expression of type `bool`.
- `typename C::dispatch_type` is a trivial type.
- `typename C::overload_types` is a tuple-like type whose element types are a non-empty set of distinct types each meeting *ProOverload*.

### § 6.2.4 *ProConvention*

A type `C` meets the *ProConvention* requirements of a type `P` if `C` meets *ProBasicConvention* and for each overload type `O` in `typename C::overload_types`:

- If `C::is_direct` is `true`, `typename C::dispatch_type` meets *ProDispatch* of `P` and `O`.
- Otherwise (`C::is_direct == false`): let `QP` be `P` with the cv and ref qualifiers of `O` (lvalue reference if `O` has no ref qualifier). The expression `*std::forward<QP>(qp)` shall be well-formed for an lvalue `qp` of type `QP`; and `typename C::dispatch_type` meets *ProDispatch* of `decltype(*std::forward<QP>(qp))` and `O`.

## § 6.2.5 *ProBasicReflection*

A type `R` meets the *ProBasicReflection* requirements if:

- `R::is_direct` is a core constant expression of type `bool`.
- `typename R::reflector_type` is a trivial type that defines a data structure describing reflected metadata.

## § 6.2.6 *ProReflection*

A type `R` meets the *ProReflection* requirements of a type `P` if `R` meets *ProBasicReflection* and the expression `typename R::reflector_type(std::in_place_type<T>)` is a core constant expression that constructs a `typename R::reflector_type`, where `T` is `P` if `R::is_direct` is `true`, otherwise `typename std::pointer_traits<P>::element_type`.

## § 6.2.7 *ProBasicFacade*

A type `F` meets the *ProBasicFacade* requirements if:

- `typename F::convention_types` is a tuple-like type containing zero or more distinct types each meeting *ProBasicConvention*.
- `typename F::reflection_types` is a tuple-like type containing zero or more distinct types each defining a reflection data structure.
- `F::max_size`, `F::max_align` are core constant expressions of type `size_t`.
- `F::copyability`, `F::relocatability`, `F::destructibility` are core constant expressions of type `constraint_level`.

## § 6.2.8 *ProFacade*

A type `F` meets the *ProFacade* requirements of a type `P` if `F` meets *ProBasicFacade* and:

- Each convention type in `typename F::convention_types` meets *ProConvention* of `P`.
- Each reflection type in `typename F::reflection_types` meets *ProReflection* of `P`.
- `F::max_size >= sizeof(P)` and `F::max_align >= alignof(P)`.

- The constraint levels impose the following requirements on P:
  - For copyability: none imposes no requirement; nontrivial requires `is_copy_constructible_v<P>`; nothrow additionally requires the copy construction is non-throwing; trivial requires `is_trivially_copy_constructible_v<P>`.
  - For relocatability (move + destroy) and destructibility the mapping is defined analogously; relocatability trivial indicates trivial relocatability.

## § 6.2.9 *ProAccessible*

A type `T` meets the *ProAccessible* requirements of types `Args...` if `typename T::template accessor<Args...>` is a nothrow-default-constructible, trivially copyable, non-final type to inject invocation or reflection functions into proxy or proxy\_indirect\_accessor specializations.

## § 6.3 Header synopsis

The synopsis below shows the new and updated declarations.

```
namespace std {

    enum class constraint_level { none, nontrivial, nothrow, trivial };

    template <template <class> class O>
        struct facade_aware_overload_t {
            facade_aware_overload_t() = delete;
        };

    template <class F>
        concept facade = see below;

    template <class P, class F>
        concept proxiable = see below;

    template <facade F>
        class proxy_indirect_accessor;

    template <facade F>
        class proxy;

    template <class D, class O, facade F, class... Args>
        see below proxy_invoke(proxy_indirect_accessor<F>& p, Args&&... args);

    template <class D, class O, facade F, class... Args>
        see below proxy_invoke(const proxy_indirect_accessor<F>& p, Args&&...
            args);

    template <class D, class O, facade F, class... Args>
        see below proxy_invoke(proxy_indirect_accessor<F>&& p, Args&&...
            args);

    template <class D, class O, facade F, class... Args>
```

```

        see below proxy_invoke(const proxy_indirect_accessor<F>&& p,
                                Args&&... args);

template <class D, class O, facade F, class... Args>
    see below proxy_invoke(proxy<F>& p, Args&&... args);

template <class D, class O, facade F, class... Args>
    see below proxy_invoke(const proxy<F>& p, Args&&... args);

template <class D, class O, facade F, class... Args>
    see below proxy_invoke(proxy<F>&& p, Args&&... args);

template <class D, class O, facade F, class... Args>
    see below proxy_invoke(const proxy<F>&& p, Args&&... args);

template <class R, facade F>
    const R& proxy_reflect(const proxy_indirect_accessor<F>& p) noexcept;

template <class R, facade F>
    const R& proxy_reflect(const proxy<F>& p) noexcept;

struct substitution_dispatch;

template <facade F>
    struct observer_facade;

template <facade F>
    using proxy_view = proxy<observer_facade<F>>;

template <facade F>
    struct weak_facade;

template <facade F>
    using weak_proxy = proxy<weak_facade<F>>;

} // namespace std

```

## § 6.4 General wording

All declarations are in namespace `std`.

### § 6.4.1 Enum class `constraint_level`

```
enum class constraint_level { none, nontrivial, nothrow, trivial };
```

For a lifetime operation (copy construction, relocation, or destruction) designated by context:

- `none`: no requirements.
- `nontrivial`: the operation is supported.
- `nothrow`: the operation is supported and is non-throwing.
- `trivial`: the operation is trivial.

## § 6.4.2 Concept facade

```
template<class F> concept facade = see below;
```

facade<F> is satisfied if and only if F meets the *ProBasicFacade* requirements.

## § 6.4.3 Concept proxiable

```
template<class P, class F> concept proxiable = see below;
```

proxiable<P, F> is satisfied if and only if F meets *ProFacade* of P. Evaluating proxiable<P, F> where P is an incomplete type whose completion could change the result is undefined behavior.

## § 6.4.4 Class template proxy\_indirect\_accessor

```
template<facade F> class proxy_indirect_accessor;
```

proxy\_indirect\_accessor<F> is an indirection helper providing accessors for the *indirect* conventions and reflections of F (those whose `is_direct` member is `false`). It has no default, copy, or move constructors. For each convention type C in `F::convention_types` with `C::is_direct == false`, if `C::dispatch_type` meets *ProAccessible* of proxy\_indirect\_accessor<F>, `C::dispatch_type`, `substituted-overload-types...`, proxy\_indirect\_accessor<F> inherits `C::dispatch_type::accessor<proxy_indirect_accessor<F>, C::dispatch_type, substituted-overload-types...>`. For each reflection type R in `F::reflection_types` with `R::is_direct == false` and whose `reflector_type` meets *ProAccessible* of proxy\_indirect\_accessor<F>, `R::reflector_type`, it inherits `R::reflector_type::accessor<proxy_indirect_accessor<F>, R::reflector_type>`.

## § 6.4.5 Class template proxy

```
template<facade F> class proxy;
```

A proxy<F> object either contains a value or does not contain a value. If it contains a value the contained type P is a pointer type such that proxiable<P, F> is `true`. The value of type P is stored within the proxy object; construction of P may perform dynamic allocation per P's semantics, but the proxy does not allocate storage for P.

Let Cs be the element types of `F::convention_types` and Rs the element types of `F::reflection_types`.

For each convention type C in Cs with `C::is_direct == true`, if `C::dispatch_type` meets *ProAccessible* of proxy<F>, `C::dispatch_type`, `substituted-overload-types...`, proxy<F> inherits the corresponding accessor. For each reflection type R in Rs with `R::is_direct == true` whose `reflector_type` meets *ProAccessible* of proxy<F>, `R::reflector_type`, proxy<F> inherits the corresponding accessor.

### § 6.4.5.1 Synopsis

```
template <facade F>
class proxy {
public:
    using facade_type = F;

    proxy() noexcept { initialize(); }
    proxy(std::nullptr_t) noexcept : proxy() {}
    proxy(const proxy&) noexcept
        requires(see below)
    = default;
    proxy(const proxy& rhs) noexcept(see below)
        requires(see below);
    proxy(proxy&& rhs) noexcept(see below)
        requires(see below);
    template <class P>
    constexpr proxy(P&& ptr) noexcept(see below)
        requires(see below);
    template <class P, class... Args>
    constexpr explicit proxy(std::in_place_type_t<P>, Args&&... args) noex-
        cept(see below)
        requires(see below);
    template <class P, class U, class... Args>
    constexpr explicit proxy(
        std::in_place_type_t<P>, std::initializer_list<U> il,
        Args&&... args) noexcept(see below)
        requires(see below);
    proxy& operator=(std::nullptr_t) noexcept(see below)
        requires(see below);
    proxy& operator=(const proxy&) noexcept requires(see below) = default;
    proxy& operator=(const proxy& rhs) noexcept(see below)
        requires(see below);
    proxy& operator=(proxy&& rhs) noexcept(see below)
        requires(see below);
    template <class P>
    constexpr proxy& operator=(P&& ptr) noexcept(see below)
        requires(see below);
    ~proxy() requires(see below) = default;
    ~proxy() noexcept(see below)
        requires(see below);

    bool has_value() const noexcept;
    explicit operator bool() const noexcept;
    void reset() noexcept(see below)
        requires(see below);
    void swap(proxy& rhs) noexcept(see below)
        requires(see below);
    template <class P, class... Args>
    constexpr P& emplace(Args&&... args) noexcept(see below) requires(see
        below);
    template <class P, class U, class... Args>
    constexpr P& emplace(std::initializer_list<U> il, Args&&... args) noex-
        cept(see below) requires(see below);

    proxy_indirect_accessor<F>* operator->() noexcept;
```

```

const proxy_indirect_accessor<F>* operator->() const noexcept;
proxy_indirect_accessor<F>& operator*() & noexcept;
const proxy_indirect_accessor<F>& operator*() const& noexcept;
proxy_indirect_accessor<F>&& operator*() && noexcept;
const proxy_indirect_accessor<F>&& operator*() const&& noexcept;

    friend void swap(proxy& lhs, proxy& rhs) noexcept(noexcept(lhs.swap(rhs)));
friend bool operator==(const proxy& lhs, std::nullptr_t) noexcept;
};

```

## § 6.4.5.2 Constructors

```
proxy() noexcept;
```

```
proxy(nullptr_t) noexcept;
```

Effects: Constructs an object that does not contain a value.

`proxy(const proxy&) noexcept` is defined as deleted unless `F::copyability == constraint_level::trivial`, in which case it is trivial and performs a bitwise copy.

`proxy(const proxy& rhs) noexcept(F::copyability == constraint_level::nothrow)` is declared when `F::copyability` is nontrivial or nothrow. Effects: if `rhs` contains a value of type `P`, initializes the contained value of the constructed object with a copy of `rhs`'s contained value; otherwise constructs an empty proxy.

`proxy(proxy&& rhs) noexcept(F::relocatability == constraint_level::nothrow)` is declared when `F::relocatability >= constraint_level::nontrivial` and `F::copyability != constraint_level::trivial`. Effects: if `rhs` contains a value, moves its contained value into the constructed object (using either a move construction of `P` or an implementation-defined trivial relocation when permitted by the constraint levels); otherwise constructs an empty proxy. Postconditions: `rhs` does not contain a value.

`template<class P> explicit(false) proxy(P&& ptr) noexcept(is_nothrow_constructible_v<decay_t<P>, P>)` participates in overload resolution only if `decay_t<P>` is not a specialization of `proxy` and is not a specialization of `in_place_type_t`. Requires: `is_constructible_v<decay_t<P>, P>` and `proxiable<decay_t<P>, F>`. Effects: constructs a contained value of type `decay_t<P>` initialized with `std::forward<P>(ptr)`.

```
template<class P, class... Args> explicit proxy(in_place_type_t<P>,
Args&&... args)
```

```
template<class P, class U, class... Args> explicit proxy(in_place_type_t<P>,
initializer_list<U> il, Args&&... args)
```

Require: the selected constructor of `P` is well-formed and `proxiable<P, F>` is `true`. Effects: constructs a contained value of type `P` direct-non-list-initialized with the arguments.

### § 6.4.5.3 Destructor

```
~proxy();
```

Effects: If the object contains a value of type P, destroys that value. The operation is conditionally trivial and/or noexcept according to `F::destructibility` and `F::relocatability`.

### § 6.4.5.4 Assignment

Copy/move assignment operators are provided analogously to the constructors, subject to the same constraint levels; detailed wording is omitted for brevity and follows existing patterns for conditionally trivial wrappers.

### § 6.4.5.5 Observers

```
explicit operator bool() const noexcept;
```

```
bool has_value() const noexcept;
```

Returns: `true` if the object contains a value, otherwise `false`.

#### 6.4.5.5.1 Indirection (`operator->` and `operator*`)

```
proxy_indirect_accessor<F>* operator->() noexcept;
```

```
const proxy_indirect_accessor<F>* operator->() const noexcept;
```

```
proxy_indirect_accessor<F>& operator*() & noexcept;
```

```
const proxy_indirect_accessor<F>& operator*() const& noexcept;
```

```
proxy_indirect_accessor<F>&& operator*() && noexcept;
```

```
const proxy_indirect_accessor<F>&& operator*() const&& noexcept;
```

Returns: `operator->()` returns a pointer to, and `operator*()` returns a (possibly cv/ref-qualified) reference or xvalue reference to, the `proxy_indirect_accessor<F>` subobject that provides accessors for all indirect conventions and reflections (those with `is_direct == false`).

Remarks: The behavior is undefined if `!has_value()`.

Notes: No value-presence check is performed; programs can test `has_value()` or compare with `nullptr` prior to calling these operators.

### § 6.4.5.6 Modifiers

```
void reset() noexcept;
```

Effects: if the object contains a value, destroys the value; afterwards the object does not contain a value.

```
template<class P, class... Args> P& emplace(Args&&... args);
```

```
template<class P, class U, class... Args> P& emplace(initializer_list<U>  
il, Args&&... args);
```

Requires: `proxiable<P, F>` is `true`. Effects: destroys any contained value, then constructs a contained value of type `P` direct-non-list-initialized from the arguments. Returns: a reference to the new contained value.

#### § 6.4.5.7 Friend functions

```
friend bool operator==(const proxy& p, nullptr_t) noexcept;
```

Returns: `!p.has_value()`.

```
friend void swap(proxy& a, proxy& b) noexcept(/* see below */);
```

Effects: Exchanges the contained values (including emptiness). The exception specification is `noexcept` if relocating or copying the contained pointer types as required is non-throwing under the constraint levels.

#### § 6.4.6 proxy\_invoke

For a function template specialization `proxy_invoke<D, 0, F>(ref, args...)`, let `0` be a type meeting *ProOverload* and `D` a type meeting *ProDispatch* of the relevant target type and `0`.

Overloads taking `proxy_indirect_accessor<F>`: Let `ptr` be the contained pointer of the associated `proxy<F>` object with the same cv/ref qualification as the parameter `ref`. Effects: `return INVOKE<R>(D(), *ptr, static_cast<Args2>(args)...) where Args2... and R are the parameter and return types of 0.`

Overloads taking `proxy<F>`: Requires: the object contains a value. Effects: `return INVOKE<R>(D(), ptr, static_cast<Args2>(args)...) where ptr is the contained value.`

#### § 6.4.7 proxy\_reflect

```
template<class R, facade F> const R& proxy_reflect(const proxy_indirect_  
t_accessor<F>& p) noexcept;
```

Effects: Returns a `const R&` direct-non-list-initialized from `in_place_type<T>` where `T` is the pointee type of the contained pointer of the associated `proxy<F>` object.

```
template<class R, facade F> const R& proxy_reflect(const proxy<F>& p)  
noexcept;
```

Requires: `p.has_value()`. Effects: Returns a `const R&` direct-non-list-initialized from `in_place_type<P>` where P is the contained pointer type.

The reference may be invalidated if the proxy is subsequently modified.

## § 6.4.8 Class `substitution_dispatch`

```
struct substitution_dispatch { substitution_dispatch() noexcept; }
```

```
template<class T> T&& operator()(T&& value) const noexcept;
```

Effects: Returns `std::forward<T>(value)`. Remarks: If T is an unqualified object type and an rvalue and the target facade substitution produces a proxy instantiation whose contained type is trivially relocatable, the implementation may relocate the storage bitwise when initializing the result proxy instead of invoking a move constructor.

### § 6.4.8.1 Class template `substitution_dispatch::accessor`

```
template<class P, class D, class... Os> struct accessor;
```

The nested class template `substitution_dispatch::accessor` participates in satisfying the *ProAccessible* requirements for direct conventions whose `dispatch_type` is `substitution_dispatch` and whose overload types return `proxy<G>` (or adapted forms in view / weak facades).

1. For any parameter pack `Os...`, the primary template `accessor<P, D, Os...>` is not constructible.
2. For any pack `Os...` with `sizeof...(Os) > 1` where each `accessor<P, D, 0>` (with 0 a single element of `Os...`) is nothrow-default-initializable, the partial specialization

```
template<class P, class D, class... Os>
requires(sizeof...(Os) > 1 && (std::is_constructible_v<accessor<P, D,
    Os>> && ...))
struct accessor<P, D, Os...> : accessor<P, D, Os>... { using acces-
    sor<P, D, Os>::operator return-type-of<Os>...; };
```

inherits all base `accessor<P, D, 0>` specializations and brings their conversion operators into scope via `using`-declarations. Here `return-type-of<0>` denotes the return type of the overload type 0. 3. For an overload type 0 of the form `proxy<F>() cv ref noex` (for some facade F and cv/ref/noexcept qualifiers), the partial specialization

```
template<class P, class D, facade F>
struct accessor<P, D, proxy<F>() cv ref noex> { operator proxy<F>() cv
    ref noex; };
```

provides an implicit conversion function `operator proxy<F>() cv ref noex`.

Effects: Let `self` denote the `proxy<F>` object that contains the accessor base subobject, with the same cv/ref qualifiers as the operator being formed. If `self.has_value()` is

`false`, returns an empty proxy<F>. Otherwise, returns the result of proxy\_invoke<substitution\_dispatch, proxy<F>() cv ref noexcept(self) (where the indicated overload type is used to select the substitution dispatch). The returned proxy contains a value substituting the underlying object; null propagation is thus guaranteed.

Remarks: Implementations may perform trivial relocation when constructing the result as permitted by the *relocatability* constraint level of the target facade.

All members not explicitly specified are exposition-only and have no further observable effect.

## § 6.4.9 observer\_facade and alias template proxy\_view

```
template<facade F> struct observer_facade;
```

For each convention C in F::convention\_types: - If C::is\_direct == `false`, include C unchanged in observer\_facade<F>::convention\_types. - Else if C::dispatch\_type is substitution\_dispatch, include a transformed convention C' whose overload set is identical to C except each overload returning proxy<G> is replaced with the same signature returning proxy\_view<G> preserving cv/ref qualifiers and `noexcept`. - Otherwise discard C.

For each reflection type R in F::reflection\_types, include R only if R::is\_direct == `false`.

observer\_facade<F>::max\_size == sizeof(void\*), max\_align == alignof(void\*), and the constraint levels are all constraint\_level::trivial.

```
template<facade F> using proxy_view = proxy<observer_facade<F>>;
```

## § 6.4.10 weak\_facade and alias template weak\_proxy

```
template<facade F> struct weak_facade;
```

weak\_facade<F>::convention\_types consists of: \* A first direct convention providing an overload proxy<F>() const noexcept named lock whose dispatch type models *ProDispatch* and whose invocation attempts to obtain a strong proxy<F> referencing the underlying object, returning an empty proxy if the object has expired. \* For each direct convention C of F whose dispatch\_type is substitution\_dispatch, a transformed convention C' identical except that each overload returning proxy<G> is replaced with an overload returning weak\_proxy<G> preserving qualifiers and `noexcept`.

No other conventions are included. weak\_facade<F>::reflection\_types is empty. The constraint constants (max\_size, max\_align, and all three constraint\_level members) are copied from F.

```
template<facade F> using weak_proxy = proxy<weak_facade<F>>;
```

## § 6.4.11 Equality and swap

For any proxy<F> objects a and b: `a == nullptr` if and only if `!a.has_value()`. Overload resolution selects the friend defined above. `swap(a, b)` exchanges their contained values (including emptiness). The complexity is `O(1)`.

# § 7 Acknowledgments

Thanks to the WG21 members who reviewed earlier revisions, especially the participants in LEWGI who asked for a clearer explanation of the type theoretic structure behind proxy. Thanks to Tian Liao (Microsoft) for the insights into the library design and open source. Thanks to Wei Chen (Jilin University), Herb Sutter, Roger Orr, Chandler Carruth, Daveed Vandevoorde, Bjarne Stroustrup, JF Bastien, Bengt Gustafsson, Chuang Li (Microsoft), Nevin Liber and Billy Baker for their valuable feedback on earlier revisions of this paper.

# § 8 Summary

proxy is a proposed <memory> facility for runtime polymorphism built purely on ordinary pointer semantics: a proxy<F> holds a pointer-like witness plus facade metadata describing operations (conventions), optional reflection, and storage / lifetime constraints. Interfaces become post-hoc, composable facades rather than baked-in base classes; concrete types stay unchanged.

Key points:

- High-performance dispatch: 0–1 dependent loads before the indirect branch (vs 2 for typical virtual), optional embedded metadata, trivial relocation when constraints allow.
- Non-intrusive: works with existing (non-hierarchical) types; no vptr injection or mandatory allocation.
- Unified ownership spectrum: strong proxy, observing proxy\_view, and weak\_proxy share the same callable surface; substitution\_dispatch enables capability downgrades without reconstruction.
- Precise contracts: named requirements (*ProOverload*, *ProDispatch*, *ProFacade*, etc.) give clear diagnostics and predictable optimization triggers (constraint levels).
- Minimal standard surface: core substrate only; helpers/factories (in the shipping library) intentionally deferred for focus and review tractability.

In short, proxy supplies a compact, extensible, production-proven substrate for runtime polymorphism: it leaves object layouts untouched, unifies ownership modes, enables post-hoc capability composition and refinement, and delivers measurable dispatch and lifetime efficiencies. It is a modern, library-only successor to traditional virtual functions.